Structures de données linéaires 1: pile et file - implémentation et interface

Complément-rappels: les tableaux (le retour)

En classe de première on a rencontré et utilisé les tableaux. En toute rigueur, ce sont des structures ayant une taille fixe et contenant des objets de même type.
Cependant, Python n'implémente pas des tableaux répondant à ces critères (du moins pas sans module spécialisé).
A la place, on trouve des structures plus complexes: les tableaux dynamiques (*les fameuses "listes de python"*) qui ont la propriété de pouvoir être agrandis, à droite.
On peut ajouter ou retirer des éléments d'une *liste de python* avec les méthodes `append` et `pop`.

```
t = [10, 20, 30]
t.append(50)
t.append(70)
print(t)

>>> [10, 20, 30, 50, 70]
t.pop()
>>> 70
print(t)
>>> [10, 20, 30, 50]
```

On peut éventuellement agrandir le tableau "par la gauche" avec la méthode insert:

```
t.insert(0, -20)
print(t)

>>> [-20, 10, 20, 30, 50]
```

On évitera autant que possible cette pratique à cause de ses médiocres performances, notamment sur les tableaux de grande taille.
Enfin, l'accès à la dernière valeur du tableau est très simple avec l'index -1:

```
print(t[-1])
>>> 50
```

Les structures de données linéaires

En informatique, pour chaque type de données, pour chaque utilisation prévue, une structure particulière de données se revèlera probablement plus adaptée qu'une autre.

Lorsque les données ne comportent pas de hiérarchie et jouent le même rôle, on utilise des **structures de données linéaires** comme les dictionnaires ou les listes. Par ailleurs, le problème de l'entrée ou de la sortie des données dans la structure peut être important. Ce qui nous amènera à distinguer ces structures linéaires entre elles.

Une première structure simple: la pile

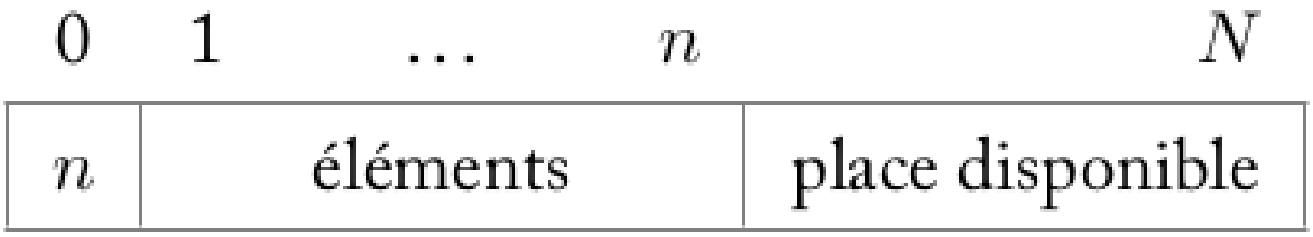
Présentation

Dans une pile d'assiettes, les manipulations **enlever** ou **déposer** une nouvelle assiette, ne peuvent se faire qu'au sommet. Une propriété remarquable est qu'une assiette ne peut être enlevée qu'après avoir enlevé toutes celles qui se trouvaient au dessus. Il s'agit d'une structure de type **LIFO**, qui signifie *Last In First Out*.

En informatique, il existe une structure de donnée appelée *pile* (*stack* en anglais) dont le fonctionnnement peut rappeler celui de la pile d'assiette. Les applications sont très nombreuses. Citons par exemple, le problème de l'annulation de la dernière action enregistrée de nombreux logiciels (le fameux `^ Ctrl` + `Z`). Il s'agit de sauvegarder les actions de l'utilisateur et de pouvoir restaurer la dernière si besoin. Il existe énormément de ressources de vulgarisation sur Internet permettant de découvrir son fonctionnement

Réalisation (implémentation) simple

Une première réalisation possible d'une pile (*bornée*) peut se faire avec un tableau (de taille fixe $N+1$). La première case du tableau (index 0) stockera le nombre n d'éléments de la pile, permettant ainsi d'avoir accès au **sommet**.




```python
N = 5 #la taille choisie est arbitraire

# Création d'un pile vide capable d'accueillir N éléments par défaut
def creer_pile(n=N):
    p = [None for _ in range(n + 1)]
    p[0] = 0
    return p
```

Les opérations d'insertions et de suppressions, qu'on nommera ici **empiler** et **dépiler**, se font au sommet.
Il est évident qu'on ne peut pas dépiler une pile vide ou empiler une pile pleine ! Il faut donc gérer ces cas. On peut éventuellement prévoir une fonction `est_pilevide`.

```python
def est_pilevide(p):
    return p[0] == 0

def depiler(p):
    n = p[0]
    assert not est_pilevide(p), "Erreur: pile vide"
    p[0] -= 1 # ne pas oublier de réajuster le sommet
    return p[n]

def empiler(p, val):
    n = p[0]
    assert n < len(p) - 1, "Erreur: pile pleine"
    n += 1 # la pile va contenir un élément de plus
    p[0] = n
    p[n] = val
```

Enfin, il est courant d'avoir une fonction `sommet` qui récupère le seul élément accessible de la pile.

```python
def sommet(p):
    assert not est_pilevide(p), "Erreur: pile vide"
    return p[p[0]]
```

Application

Donner la situation de la pile après chacune des opérations suivantes:

```
p = creer_pile()
empiler(p, 5)
empiler(p, 15)
empiler(p, 25)
depiler(p)
```

Quelle valeur se trouve au sommet de la pile? Le vérifier.

On peut aussi réaliser une pile (*non bornée*) avec des *listes de python* et leurs méthodes `append` et `pop`.

A faire

Compléter les fonctions `depiler`, `empiler` et `sommet`.

```
def creer_pile():
    return []

def est_pilevide(p):
    return len(p) == 0

def depiler(p):
    # A compléter
    pass

def empiler(p, val):
    # A compléter
    pass

def sommet(p):
    # A compléter
    pass
```

```
p = creer_pile()
empiler(p, 5)
empiler(p, 15)
empiler(p, 25)
depiler(p)
print(sommet(p))
depiler(p)
print(sommet(p))
```

```
15
5
```

A retenir

Les opérations réalisables sur une pile peuvent donner lieu à des réalisations (on dit aussi implémentations) différentes.

Une deuxième structure: la file

Présentation

La file est une structure qui diffère de la pile dans le sens où l'insertion et la suppression ne se font pas du même côté (penser à une *file d'attente* devant un guichet). Ces opérations sont nommées **enfiler** et **défiler**. Les files sont par exemple utilisées dans les gestionnaires d'impression (des imprimantes). Il s'agit d'une structure de type **FIFO**, pour *First In First Out*.

Implémentation

Une implémentation triviale peut être réalisée avec des *listes de python*.

A faire

Compléter le code suivant. *Coup de pouce*: utiliser les méthodes `append` , `pop` et `insert`.

```python
def creer_file():
    return []

def est_filevide(f):
    return len(f) == 0

def defiler(f):
    # A compléter
    pass

def enfiler(f, val):
    # A compléter
    pass
```

Vérifier que l'on a bien une structure de file en exécutant la cellule suivante

```python
f = creer_file()
enfiler(f, 5)
enfiler(f, 15)
enfiler(f, 25)
while not est_filevide(f):
    print(defiler(f), end=' ')
```

```
5 15 25
```

```python
defiler(f)
```

```
---------------------------------------------------------------------------

AssertionError                            Traceback (most recent call last)

<ipython-input-66-cbba2ba98b42> in <module>
----> 1 defiler(f)


<ipython-input-64-bfde421bb079> in defiler(f)
      7 def defiler(f):
      8     #A compléter
----> 9     assert not est_filevide(f), "Erreur: file vide"
     10     return f.pop()
     11


AssertionError: Erreur: file vide
```

Une autre réalisation est possible en utilisant 2 piles qu'on nommera `p_in` et `p_out`. Les opérations `enfiler` et `défiler` sont détaillées ci dessous:

enfiler

```
empiler sur p_in.
```

defiler

```
Si p_out est vide:
    Tant que p_in n'est pas vide:
        depiler p_in et empiler cet élément sur p_out
depiler p_out
```

A faire

Compléter l'implémentation d'une file qui utilise deux piles, dans la cellule suivante.

```python
def creer_file():
    return (creer_pile(), creer_pile())

def enfiler(f, val):
    p_in = f[0]
    # A compléter
    pass

def defiler(f):
    assert not est_filevide(f), "Erreur: file vide"
    # A compléter
    pass

def est_filevide(f):
    p_in, p_out = f[0], f[1]
    return est_pilevide(p_in) and est_pilevide(p_out)
```

```python
#Cellule de tests à exécuter
f = creer_file()
enfiler(f, 5)
enfiler(f, 15)
enfiler(f, 25)
while not est_filevide(f):
    print(defiler(f), end=' ')
```

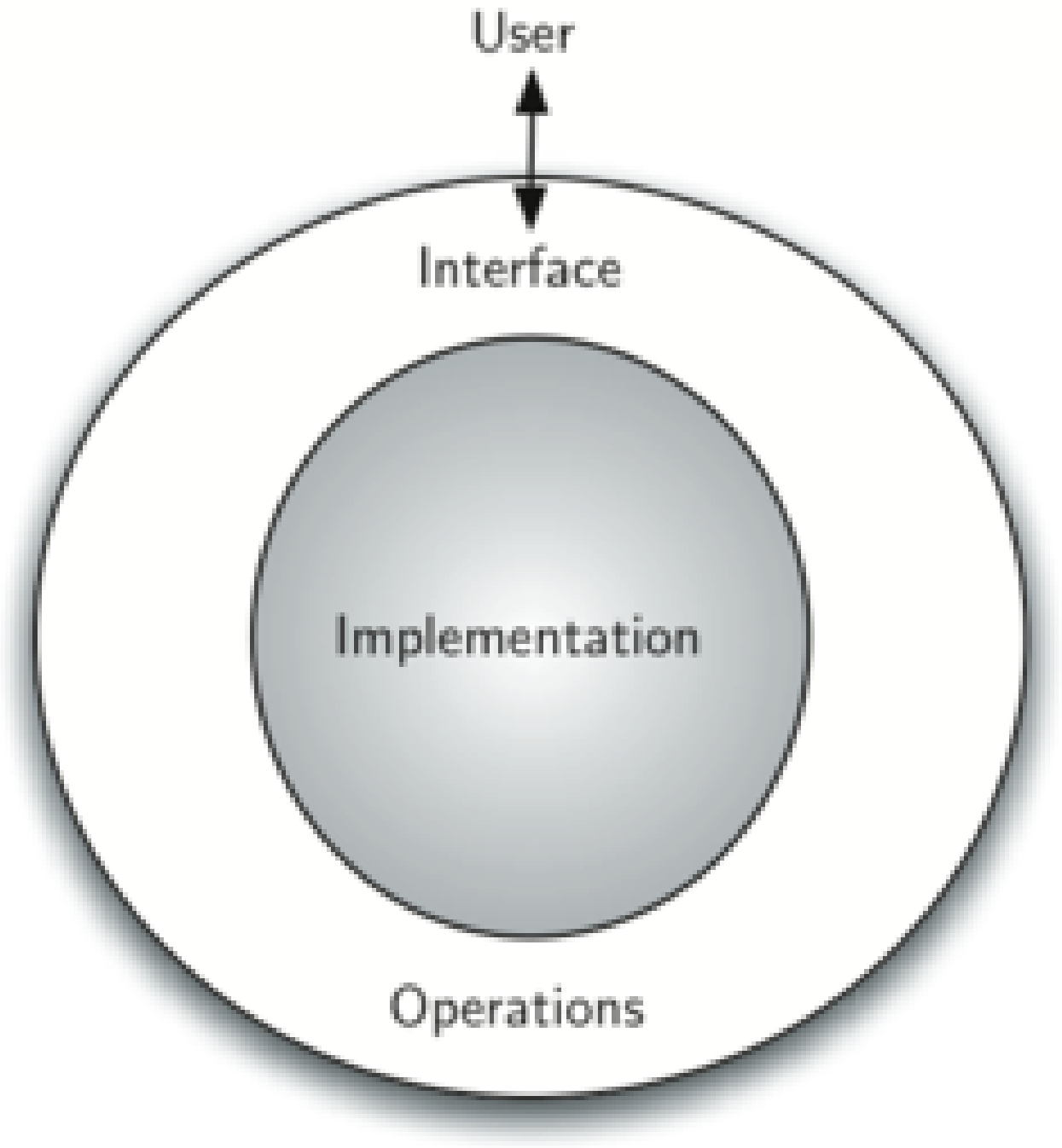
```python
defiler(f)
```

```
---------------------------------------------------------------------------

AssertionError                            Traceback (most recent call last)

<ipython-input-70-cbba2ba98b42> in <module>
----> 1 defiler(f)


<ipython-input-68-6de1afd1571e> in defiler(f)
      7
      8 def defiler(f):
----> 9     assert not est_filevide(f), "Erreur: file vide"
     10     p_in, p_out = f[0], f[1]
     11     if est_pilevide(p_out):


AssertionError: Erreur: file vide
```

Comme pour la pile, même si les implémentations d'une file peuvent être très différentes les fonctionnalités qu'elles offrent sont identiques.

Vers une abstraction des structures de données

Les exemples précédents ont montré qu'on pouvait définir **une structure de données comme étant une description logique de la façon dont on voit ces données et des opérations permises**.
D'un point de vue utilisateur (*c'est-à-dire le programmeur*), manipuler une structure de données c'est se préoccuper de ce que les données représentent et utiliser les opérations autorisées **sans se soucier de leur implémentation**.



Les détails de mise en oeuvre des structures de données étant cachés à l'utilisateur, celui-ci accède à ces objets par le biais d'une **interface** (ensemble des opérations permises et d'une description informelle de leur effet).

Voici par exemple, les spécifications de la structure de données **pile** précedente:

Struture de données Pile

Opérations	Description
creer_pile(n)	Créé une pile vide de taille n optionnelle
est_pilevide(p)	retourne un booléen correspondant à l'état de la pile p.
empiler(p, val)	empile val dans la pile p.
depiler(p)	renvoie le sommet de la pile p et le supprime de celle-ci
sommet(p)	renvoie le sommet de la pile p, sans le supprimer

Cette liste de spécifications est souvent appelée **API** pour **A**pplication **P**rogramming **I**nterface.